2. Design

2.1 Explanation for way of working

Based on Pandeli and Wayne's analysis of the clients brief and requirements, I considered their requirements table and IPO table and used some analytical skills of my own to use the data and information they gathered to create 3 designs that will help aid Henry create the Flowgarithm and final algorithm that will run the program to ensure that it works.

I will first need to decompose the analysis so that it is easy to understand and make it easier to create a functional and correct design that will help Henry create the program. I will make a subroutine list, a data plan, an array list and what each array holds inside it so that the brief is fully decomposed and simplified so that the design can be made to be easily understood

I will make a simple flowchart that should be easy to understand, this will help the client understand how the actual program is meant to run and will run, and it is important to keep the flowchart simple so that the clients aren't confused since they most likely won't be well versed in pseudo code and actual programming language and syntax.

I will make a relatively simple pseudo code that will be easy and simple to understand for a programmer so that Henry will be able to interpret it easily and port it over onto the actual Flowgarithm software that will run the program. The pseudo code won't be the actual final code and will only be a base template that will help Henry understand what the code is meant to look and function like, and it isn't meant to be a replacement for the actual functioning code.

I will then make a test plan design, so that when the final code is created, it can be tested and probed by Haneef to ensure that the program functions as intended and is able to run without any logic errors, syntax errors or runtime and performance errors. Creating this table will allow Haneef to easily do all these tests and create tests logs with the expected outcome, actual outcome, and other possible notes that could be made

2.2 Subroutine list

This is the list of subroutines that the program will need to run and function and having a list of all the subroutines allows us to fully decompose and understand the clients brief and requirements so that when we create the designs and final program, we haven't missed any details that were mentioned in the design brief

Name	Type	Takes in	Gives back / does
main( )	Module	Nothing	Sets up the variables and arrays, then calls the other subroutines in order
getCustomerDetails(customerDetails[ ])	Module	customerDetails array (empty)	Asks for and fills in the name, address and phone number
getNumPizzas( )	Function (Integer)	Nothing	Returns the number of pizzas (1 to 6). Shows an error and asks again if invalid
pizzaDetails(numOfPizza, sizeOfPizzas[ ], numOfToppings[ ])	Module	Number of pizzas and the two empty arrays	Goes through each pizza, fills the arrays with its size cost and topping cost, and shows the cost of that pizza
setSizeOfPizzas(count)	Function (Real)	The pizza number (count)	Asks for the size (1, 2 or 3) until valid. Returns the standard cost of that size
setToppings(count)	Function (Real)	The pizza number (count)	Asks for the number of extra toppings until valid. Returns the topping cost

checkDelivery( )	Function (Boolean)	Nothing	Asks if the order is being delivered. Returns True or False
displayBill(numOfPizza, sizeOfPizza[ ], numOfToppings[ ], isDelivery)	Module	Number of pizzas, the two filled arrays and isDelivery	Adds up all the costs, takes off the 10% discount if over £20, adds £2.50 if delivered, and shows the discount and order total

2.3 Data plan

Arrays are the method of storing data we have been told to use so I created a data plan, so that we can understand all the data that the program will be using, what type of data it is and what information is stored inside the data, this allows us to create an easy-to-follow design when developing the final program.

To explain, arrays start counting from 0, so PizzaSize[0] is pizza 1 and PizzaSize[5] is pizza 6. there are only 6 cells because the max number of pizzas that can be ordered it 6

Single values

Name	Type	Holds	Set by	Used by
numOfPizza	Integer	How many pizzas are in the order (1 to 6)	getNumPizzas	pizzaDetails, displayBill
isDelivery	Boolean	True if the order is being delivered	checkDelivery	displayBill

Arrays

Array	Type	What each cell holds	Written by	Used by
customerDetails[3]	String	One cell per detail (see below)	getCustomerDetails	Nothing yet
sizeOfPizzas[6]	Real	Standard cost of each pizza's chosen size (3.25, 5.50 or 7.15)	pizzaDetails	displayBill

numOfToppings[6]	Real	Extra topping cost of each pizza	pizzaDetails	displayBill

customerDetails[3]	Holds
[0]	Customer name
[1]	Customer address
[2]	Phone number

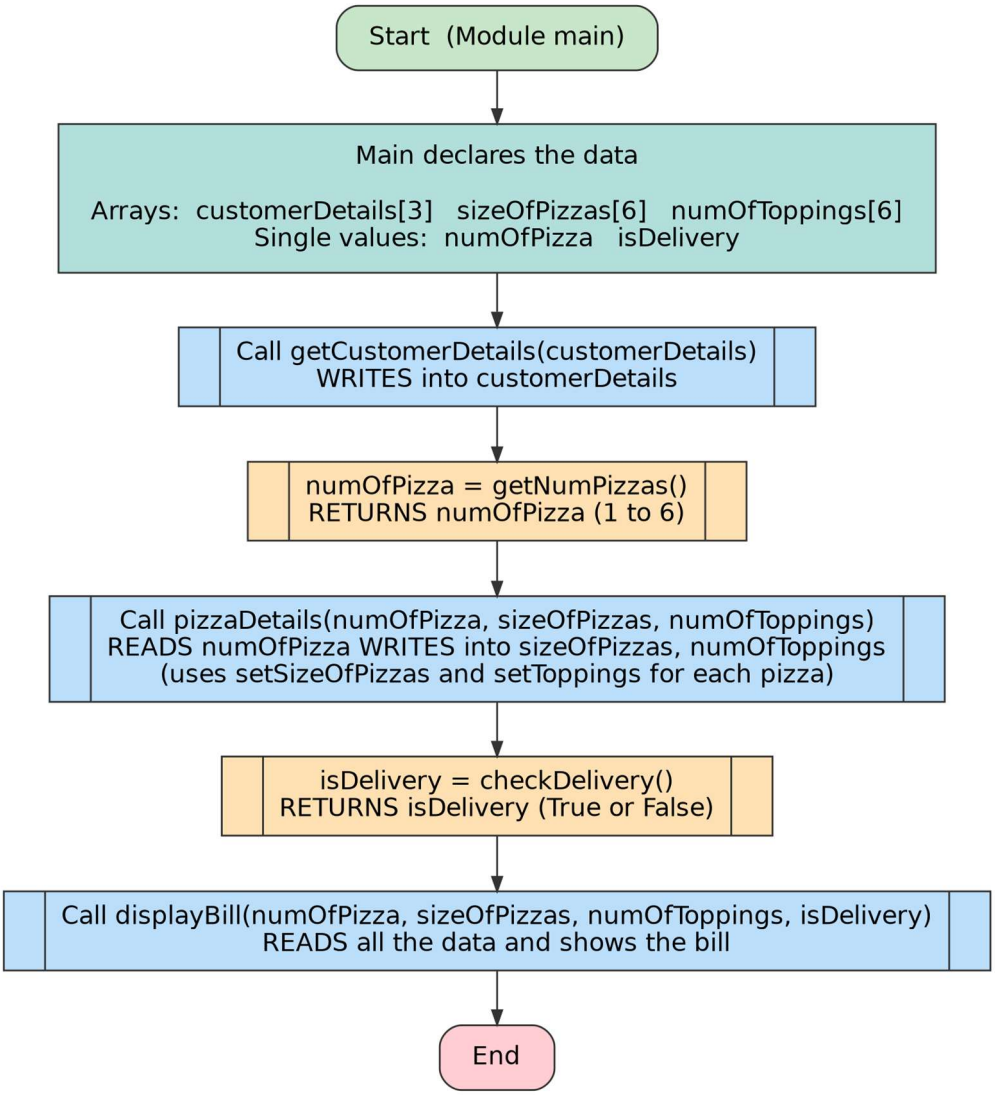
Working variables inside the subroutines

Subroutine	Working variables	Used for
getCustomerDetails	assign[3], i	Labels used in the prompts, and a loop counter
getNumPizzas	noOfPizzas	The number typed in, checked until it is 1 to 6
pizzaDetails	count	Counts through the pizzas
setSizeOfPizzas	sizeVal, label, cost	The size chosen, its name, and its cost
setToppings	val, cost	The toppings chosen and their cost
checkDelivery	check	The delivery answer
displayBill	sum, preSum, i	Running total, the subtotal before discount, and a loop counter

2.4 Main flow sketch

I used draw.io to create a flow chart that would be easily understandable for the client. The client doesn't need to understand how the program works, just what information is being used and accessed in the subroutines

This flowchart clearly explains where and when each subroutine is used, why it's being called and what it does. I was struggling to export the flowchart into this word document, so I just screenshotted the flowchart instead.



2.5 Requirements check

This is just requirements check to see that all the requirements that we analysed before are met and if a subroutine was created for the requirement, this helps us make sure that we haven't missed or forgot any client requirements and details

Req.	What the program must do	Subroutine	Where the data is
What staff enter			
R1	Customer name, address and phone number	getCustomerDetails	customerDetails[3]
R2	Number of pizzas ordered	getNumPizzas	numOfPizza
R3	Size of each pizza	setSizeOfPizzas (called by pizzaDetails)	sizeOfPizzas[6]

R4	Extra toppings for each pizza	setToppings (called by pizzaDetails)	numOfToppings[6]
R5	Whether delivery is required	checkDelivery	isDelivery
What the bill must show			
R6	Customer details	displayBill	customerDetails[3]
R7	Cost of each pizza	pizzaDetails	sizeOfPizzas[6], numOfToppings[6]
R8	Standard pizza cost and extra topping cost	displayBill	sizeOfPizzas[6], numOfToppings[6]
R9	Order subtotal before discount and delivery	displayBill	preSum
R10	Any discount applied	displayBill	sum, preSum
R11	Any delivery charge	displayBill	sum
R12	Final total cost of the order	displayBill	sum
Business rules			
R13	Minimum 1 and maximum 6 pizzas	getNumPizzas	numOfPizza
R14	Small £3.25, medium £5.50, large £7.15	setSizeOfPizzas	sizeOfPizzas[6]
R15	Toppings: 1 = £0.75, 2 = £1.35, 3 = £2.00, 4 or more = £2.50	setToppings	numOfToppings[6]
R16	10% discount if subtotal is over £20 before delivery	displayBill	sum, preSum
R17	£2.50 delivery charge if delivery is requested	displayBill	sum

2.6 Pseudocode

This is the Gaddis pseudocode for the program that was generated automatically by FLowgarithm, when the program was finished, this would let programmers read the program easily and understand what is happening in the program and how it works, it would allow them to edit and make changes to the program and translate the pseudocode into different programming languages like Python, Java, C++, etc.

```
Module main()
```

```
    // Setup Variables
    Declare String customerDetails[3]
    Declare Integer numOfPizza
    Declare Real sizeOfPizzas[6]
    Declare Real numOfToppings[6]
    Declare Boolean isDelivery

    // One Vars are set, Call GetCustomerDetails Subroutine.
    Call getCustomerDetails(customerDetails)

    // This sets the variable of 'NumOfPizza' using the 'getNumPizzas' subroutine
    Set numOfPizza = getNumPizzas()

    // Subroutine that sets the Pizza Size and Toppings
    Call pizzaDetails(numOfPizza, sizeOfPizzas, numOfToppings)
    Set isDelivery = checkDelivery()

    // Display Bill
    Call displayBill(numOfPizza, sizeOfPizzas, numOfToppings, isDelivery)
End Module

Function Boolean checkDelivery ()
    Declare Boolean check

    Display "Is this order going to be delivered? True or false only please."
    Input check

    Return check
End Function

Module displayBill (Integer numOfPizza, Real sizeOfPizza[], Real numOfToppings[], Boolean isDelivery)
    Declare Real sum
    Declare Real preSum

    Set sum = 0.0

    // The Do Loop itterates through each pizza size and topping and adds it to the sum
    Declare Integer i

    Set i = 0
    Do
        Set sum = sum + sizeOfPizza[i]
        Set sum = sum + numOfToppings[i]
        Set i = i + 1
```

```
  While numOfPizza > i

  // Checks for discount if sum > 20 before delivery
  Set preSum = sum
  If sum > 20 Then
    Display "Discount has been added to order"
    Set sum = sum * 0.9
  Else
    Display "Discount not earned for order"
  End If
  Display "Discount Ammount = £", preSum - sum

  // Checks if delivery has been selected
  If isDelivery == True Then
    Set sum = sum + 2.5
  End If
  Display "Order Subtotal = £", sum
End Module

Module getCustomerDetails (String customerDetails[])
  // This subroutine sets the details as such:
  // CustomerDetails[0]= Name
  // CustomerDetails[1]= Address
  // CustomerDetails[2]= Phone Number
  Declare String assign[3]

  Set assign[0] = "Name"
  Set assign[1] = "Address"
  Set assign[2] = "Phone Num"

  // Settup a for loop that itterates through customer details gathering the data for each feild recursively
  Declare Integer i

  For i = 0 To 2
    Display "Enter ", assign[i], "  :"
    Input customerDetails[i]
  End For
End Module

Function Integer getNumPizzas ()
  Declare Integer noOfPizzas

  Set noOfPizzas = 0
  Display "Enter number of pizzas(max 6)"
```

```
  While noOfPizzas < 1 OR noOfPizzas > 6
    Input noOfPizzas
    If noOfPizzas < 1 OR noOfPizzas > 6 Then
      Display "Error, must order minimum 1 and maximum 6 pizzas"
    Else
      Display "Ok, you've selected ", noOfPizzas, " pizzas."
    End If
  End While

  Return noOfPizzas
End Function

Module pizzaDetails (Integer numOfPizza, Real sizeOfPizzas[], Real numOfToppings[])
  Declare Integer count

  Set count = 0

  // Iterative Do loop that goes though each pizza's details
  Do

    // Function that sets the size of pizza
    Set sizeOfPizzas[count] = setSizeOfPizzas(count)

    // Function that sets number of toppings
    Set numOfToppings[count] = setToppings(count)
    Display "The cost for pizza ", count + 1, " is: £", numOfToppings[count] + sizeOfPizzas[count]
    Set count = count + 1

    // keep running till count reaches the number of pizzas
  While count < numOfPizza
End Module

Function Real setSizeOfPizzas (Integer count)
  Declare Integer sizeVal
  Declare String label
  Declare Real cost

  Set cost = 0
  Display "Select a size pizza you would like for pizza no", count + 1
  Do
    Display "1 = Small 2 = Medium 3 = Large"
    Input sizeVal
    If sizeVal > 3 OR sizeVal < 1 Then
      Display "Error: size must be within range"
```

```
    Else
      If sizeVal == 1 Then
        Set cost = cost + 3.25
        Set label = "small"
      Else
        If sizeVal == 2 Then
          Set cost = cost + 5.5
          Set label = "medium"
        Else
          Set cost = cost + 7.15
          Set label = "large"
        End If
      End If
      Display "Ok for pizza ", count + 1, " the size is: ", label, "."
    End If
  While sizeVal > 3 OR sizeVal < 1

  Return cost
End Function

Function Real setToppings (Integer count)
  Declare Integer val
  Declare Real cost

  Display "Select how many extra toppings for pizza no", count + 1
  Do
    Input val
    If val < 0 Then
      Display "Error, You cant have negative toppings!"
    Else
      If val >= 4 Then
        Set cost = 2.5
      Else
        If val == 3 Then
          Set cost = 2.0
        Else
          If val == 2 Then
            Set cost = 1.35
          Else
            Set cost = 0.75
          End If
        End If
      End If
    End If
    Display "Ok you've selected ", val, " extra toppings"
```

```
    While val < 0

    Return cost
End Function
```

Since Henry will be creating the program in flowgarithm, I didn't use any indentation as I believe that would confuse him and wasn't necessary since flowgarithm already does all the programming and syntax in the background based on the code that was created in the flow chart, I believe that this pseudocode will help guide Henry in creating the final program and should be basic enough to understand and follow when creating the final

2.7 Test table plan

This is the template for the test table that will be used when the final program is created. We're only doing simple black box testing, so I only made 3 tables, one for normal testing, boundary testing and extreme testing

Test table

Normal data

Test no.	Description	Input data	Expected result	Actual result	P / F

Boundary data

Test no.	Description	Input data	Expected result	Actual result	P / F

Extreme data

Test no.	Description	Input data	Expected result	Actual result	P / F